

**SLOVENSKÁ TECHNICKÁ UNIVERZITA V
BRATISLAVE
FAKULTA ELEKTROTECHNIKY A INFORMATIKY**

Semestrálny projekt

**IoT systém pre monitorovanie a vizualizáciu svetelnej
hudby**

Študent: Kirill Ahaponau

Rok: 2026

1. Úvod

Cieľom projektu bolo vytvoriť IoT systém, ktorý umožňuje monitorovanie zvukového signálu v reálnom čase a jeho následnú vizualizáciu pomocou RGB LED pásu a webovej aplikácie. Projekt využíva platformu Arduino Nano ako hlavný hardvérový prvok pre spracovanie zvukového signálu a Raspberry Pi OS vo virtuálnom prostredí VirtualBox ako serverovú časť.

Systém funguje ako digitálna svetelná hudba (Color Music). Mikrofón sníma hudobný signál, Arduino analyzuje zvukové frekvencie pomocou FFT algoritmu a následne riadi RGB LED pás WS2812B. Súčasne Arduino odosiela údaje cez sériovú komunikáciu do Python Flask servera.

Webová aplikácia umožňuje:

- spustenie a inicializáciu systému,
- monitorovanie hodnôt v reálnom čase,
- zobrazovanie grafov,
- zobrazovanie ciferníkov,
- archiváciu údajov,
- nastavovanie parametrov systému,
- vzdialené ovládanie systému cez webové rozhranie.

Projekt spĺňa princíp Internet of Things (IoT), pretože umožňuje komunikáciu medzi reálnym hardvérom a webovým klientom cez server.

IoT RGB Sound Visualizer

127.0.0.1:5000

IoT RGB Sound Visualizer System

System control

Open

Start

Stop

Close

Status: opened=true, monitoring=false
Session: 0
Message: OPEN: Arduino connected on /dev/ttyUSB0
Serial port: /dev/ttyUSB0

Monitoring / control parameters

Sensitivity: 1.0

Brightness: 200

Mode: Sound monitor

Save parameters

1. Numeric data

2. Live graphs

3. Gauges

4. Database archive

5. File archive

Live numerical output

Press Open and Start...

Webová aplikácia po kliknutí na tlačidlo OPEN

2. Architektúra systému

Projekt využíva klient-server architektúru.

Tok údajov:

Mikrofón → Arduino → Serial USB → Flask server → Web klient.

Arduino:

- spracúva zvuk,
- riadi LED pás,
- odosiela údaje.

Flask server:

- prijíma údaje,
- archivuje údaje,
- poskytuje údaje webovému klientovi.

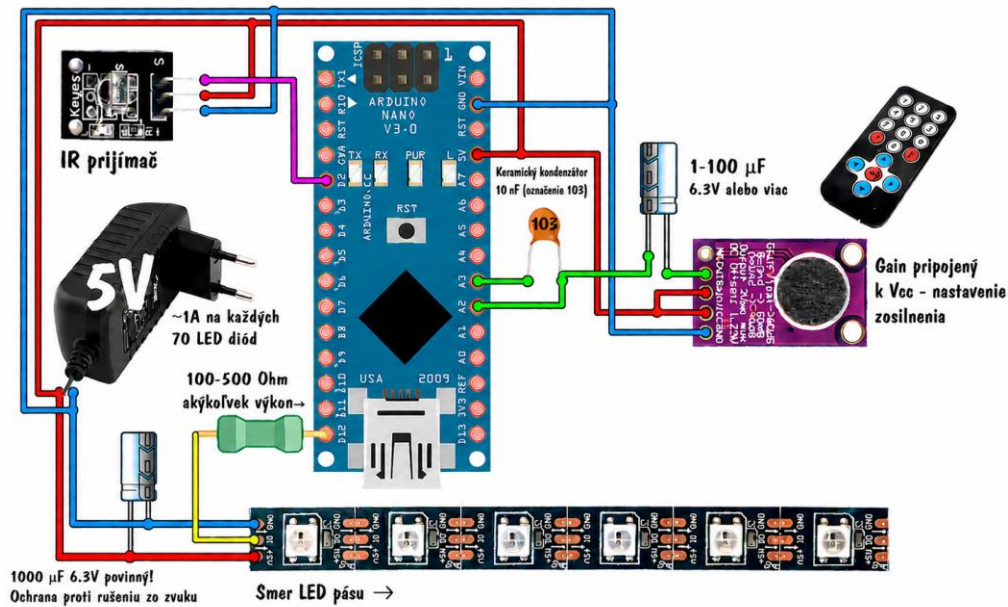
Web klient:

- vizualizuje údaje,

- umožňuje ovládanie systému.

Projekt spĺňa IoT princíp:

- reálne senzory,
- webová komunikácia,
- vzdialené monitorovanie,
- webové ovládanie.



Bloková schéma systému

3. Hardvérová realizácia

Hardvérová časť systému pozostáva z niekoľkých komponentov:

- Arduino Nano,
- RGB LED pás WS2812B,
- mikrofón MAX4466,
- IR prijímač,
- napájací zdroj 5 V,
- Raspberry Pi OS vo VirtualBoxe.

Arduino Nano zabezpečuje:

- snímanie analógového zvukového signálu,
- spracovanie zvuku,
- FFT/FHT analýzu frekvencií,
- riadenie LED pásu,

- komunikáciu s Flask serverom.

LED pás WS2812B umožňuje:

- individuálne adresovanie LED,
- vytváranie animácií,
- zmenu farieb podľa frekvenčného spektra.

Systém obsahuje viacero režimov:

- VU meter,
- rainbow visualizer,
- spectrum analyzer,
- stroboscope,
- running frequencies,
- advanced spectrum analyzer.



Puzdro zariadenia s Arduino Nano + LED pásom WS2812B + mikrofónom MAX4466

4. Serverová časť systému

Serverová časť bola implementovaná v jazyku Python pomocou Flask frameworku. Flask server zabezpečuje komunikáciu medzi Arduino a webovým klientom.

Hlavné úlohy servera:

- prijímanie údajov zo sériového portu,
- spracovanie JSON paketov,
- archivácia údajov,
- odosielanie údajov klientovi,
- riadenie systému cez HTTP požiadavky.

Použité knižnice:

- Flask,
- pyserial,
- sqlite3,
- csv,
- threading,
- json.

Arduino odosiela údaje vo forme JSON:

```
{  
  "sound": hodnota,  
  "freq": hodnota,  
  "mode": hodnota,  
  "brightness": hodnota,  
  "on": hodnota  
}
```

Server prijaté údaje:

- zobrazí vo webovom rozhraní,
- uloží do SQLite databázy,
- uloží do CSV súboru.

Flask aplikácia bola spúšťaná pomocou:
python3 app.py

```
kiri-aga@raspberrypi: ~/Downloads
File Edit Tabs Help
kiri-aga@raspberrypi:~ $ cd ./Downloads/
kiri-aga@raspberrypi:~/Downloads $ sudo python3 app.py
WebSocket transport not available. Install eventlet or gevent and gevent-websocket for improved performance.
* Serving Flask app "app" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)
127.0.0.1 - - [14/May/2026 20:22:11] "GET /socket.io/?EI0=4&transport=polling&t=Pudk0Y3 HTTP/1.1" 200 -
127.0.0.1 - - [14/May/2026 20:22:11] "POST /socket.io/?EI0=4&transport=polling&t=Pudk0YD&sid=UyrX0h52tTXf0fGi
AAAA HTTP/1.1" 200 -
127.0.0.1 - - [14/May/2026 20:22:11] "GET /socket.io/?EI0=4&transport=polling&t=Pudk0YE&sid=UyrX0h52tTXf0fGiA
AAA HTTP/1.1" 200 -
```

Flask server terminal s bežiacou aplikáciou

5. Klientská webová aplikácia

Klientská časť systému bola vytvorená pomocou HTML, CSS a JavaScript.

Webové rozhranie obsahuje:

- tlačidlo Open,
- tlačidlo Start,
- tlačidlo Stop,
- tlačidlo Close,
- grafy v reálnom čase,
- ručičkové ukazovatele,
- numerické zobrazenie hodnôt,
- ovládacie prvky.

Funkcia tlačidiel:

Open:

- inicializuje systém,
- nadviaže komunikáciu,
- aktivuje senzory.

Start:

- spustí monitorovanie údajov,

- aktivuje grafy a ciferníky.

Stop:

- zastaví monitorovanie,
- zastaví prenos údajov.

Close:

- ukončí komunikáciu,
- deaktivuje systém.

Grafy zobrazujú:

- hodnotu zvuku,
- frekvenčné údaje,
- jas LED systému.

Ciferníky zobrazujú:

- aktuálnu úroveň zvuku,
- frekvenčné zaťaženie systému.

IoT RGB Sound Visualizer System

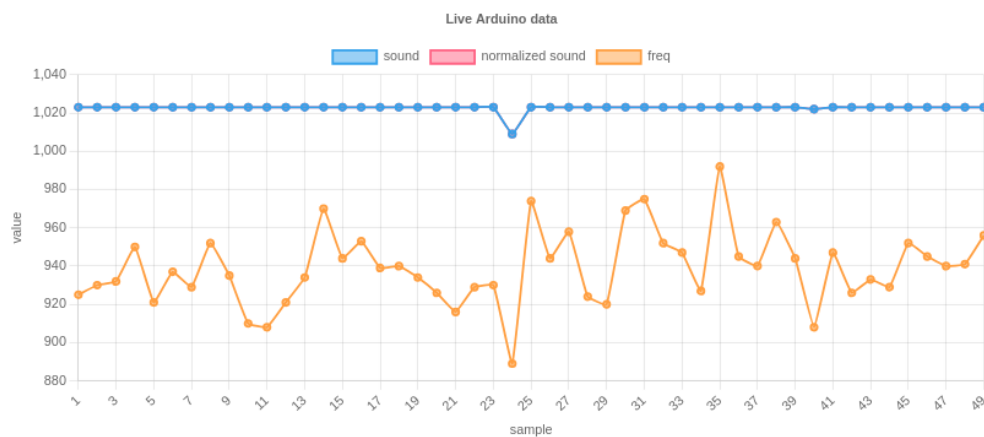
System control

Status: opened=true, monitoring=true
Session: 1
Message: START: monitoring session 1
Serial port: /dev/ttyUSB0

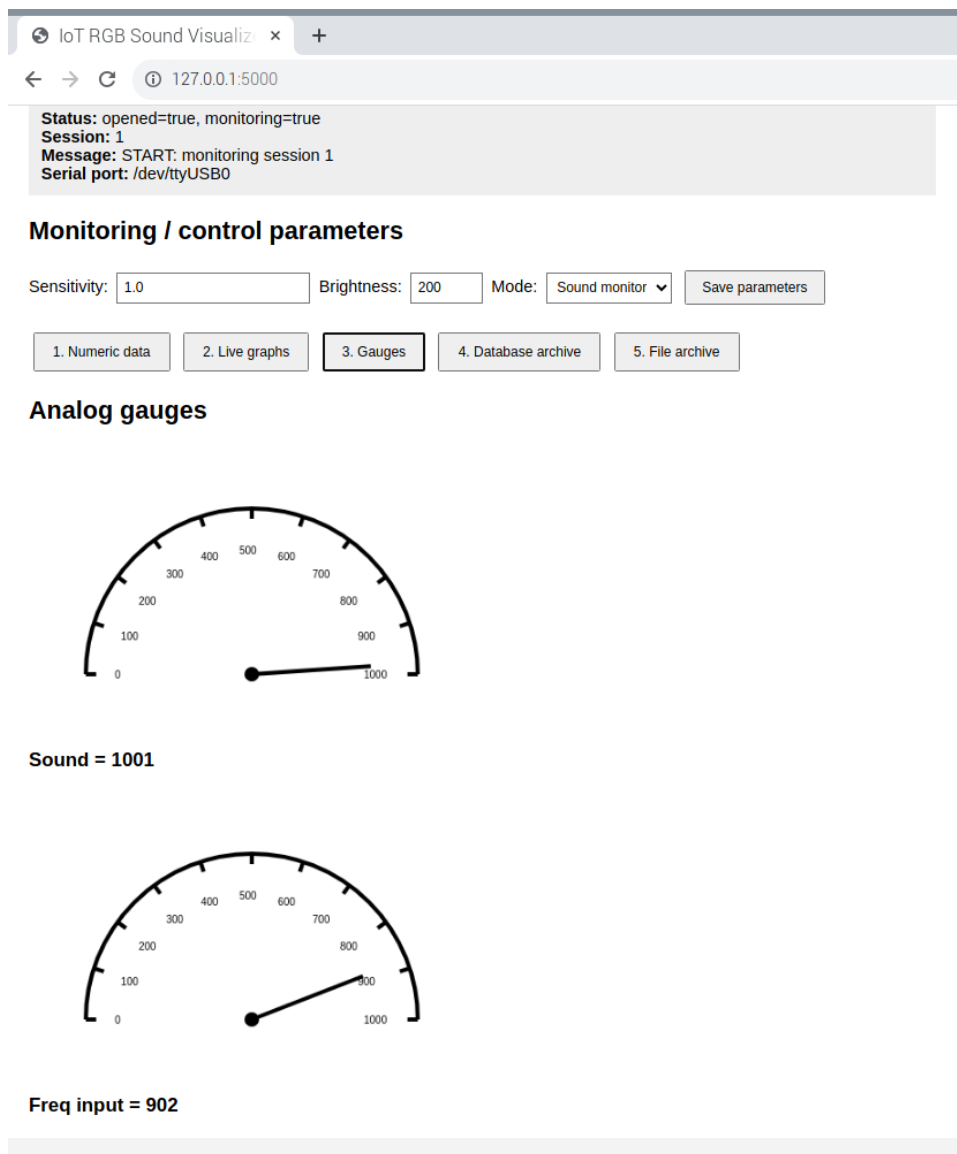
Monitoring / control parameters

Sensitivity: Brightness: Mode:

Live graph



Graf v reálnom čase



Ručičkový ukazovateľ

6. Databáza a archivácia údajov

Projekt podporuje archiváciu údajov dvoma spôsobmi.

1. SQLite databáza:

Do databázy sa ukladajú:

- hodnoty zvuku,
- frekvenčné údaje,
- jas LED,
- aktuálny režim,
- časová značka.

Výhodou databázy je:

- možnosť neskoršieho spracovania,
- jednoduché filtrovanie údajov,
- trvalé uloženie dát.

2. CSV súbor:

Údaje sa ukladajú aj do CSV súboru, čo umožňuje:

- otvorenie v Microsoft Excel,
- jednoduché grafické spracovanie,
- export údajov.

Archivácia je vykonávaná automaticky počas monitorovania.

SQLite database archive

Load sessions

```
[
  {
    "count": 78,
    "end": "2026-05-14 20:26:29",
    "session_id": 2,
    "start": "2026-05-14 14:42:26"
  },
  {
    "count": 1073,
    "end": "2026-05-14 20:26:13",
    "session_id": 1,
    "start": "2026-05-14 14:38:06"
  }
]
```

Session ID:

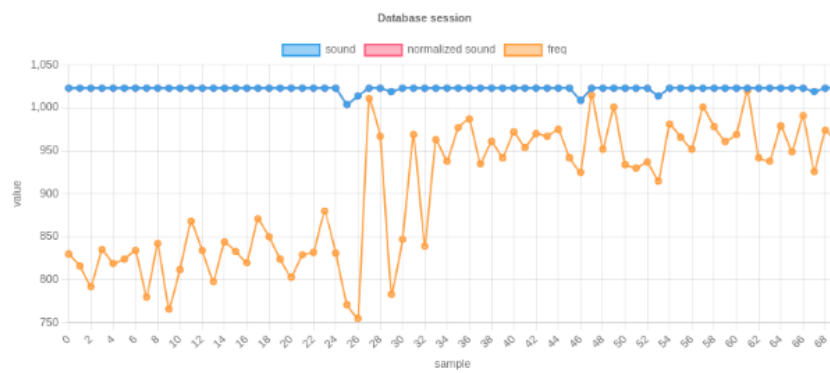
2

Load from database

Numerical output from database

```
{
  "data": [
    {
      "brightness": 200,
      "freq": 830,
      "mode": 2,
      "normalized_sound": 1023,
      "on": 1,
      "sensitivity": 1,
      "sound": 1023,
      "timestamp": "2026-05-14 14:42:26",
      "web_mode": "LED visualizer",
      "x": 0
    },
    {
      "brightness": 200,
      "freq": 816,
      "mode": 2,
      "normalized_sound": 1023,
      "on": 1
    }
  ]
}
```

Graph from database



SQLite databáza

CSV file archive

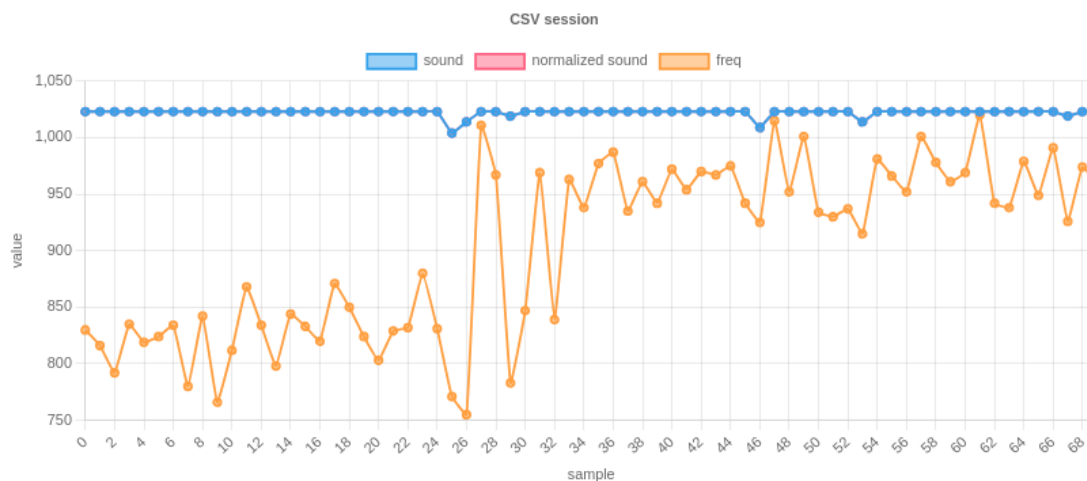
Session ID:

[Load from CSV file](#)

Numerical output from file

```
{
  "data": [
    {
      "brightness": 200,
      "freq": 830,
      "mode": 2,
      "normalized_sound": 1023,
      "on": 1,
      "sensitivity": 1,
      "sound": 1023,
      "timestamp": "2026-05-14 14:42:26",
      "web_mode": "LED visualizer",
      "x": 0
    },
    {
      "brightness": 200,
      "freq": 816,
      "mode": 2,
      "normalized_sound": 1023,
```

Graph from file



CSV exportovaný súbor

7. Komunikačný protokol

Komunikácia medzi Arduino a Raspberry Pi je realizovaná pomocou Serial USB komunikácie.

Podporované príkazy:

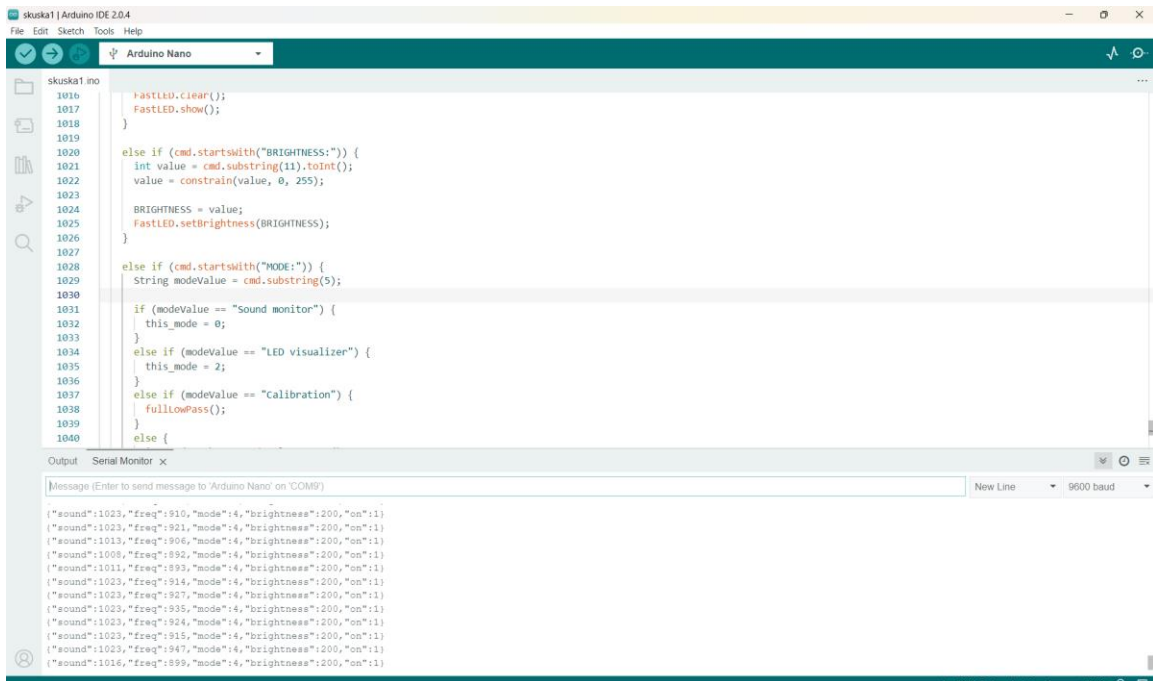
- OPEN,
- START,
- STOP,
- CLOSE,
- BRIGHTNESS,
- MODE.

Arduino reaguje na tieto príkazy a zároveň odosiela telemetrické údaje.

Prenos údajov prebieha každých 200 ms.

Použitý bol JSON formát, pretože:

- je jednoduchý,
- ľahko čitateľný,
- jednoducho spracovateľný v Pythone aj JavaScripte.



The screenshot shows the Arduino IDE interface. The top pane displays a C++ sketch for an Arduino Nano. The sketch includes a `setup` function that initializes a serial port at 9600 baud and a `FastLED` library. The `loop` function contains a series of `if` statements that respond to specific commands sent via the serial port. These commands include `BRIGHTNESS` (to set the brightness of the LEDs), `MODE` (to switch between different modes like 'Sound monitor', 'LED visualizer', and 'calibration'), and `fullLowPass` (to apply a low-pass filter). The bottom pane shows the Serial Monitor, which displays the output of the sketch. The output consists of a series of JSON-formatted strings, each representing a data point. Each string contains the frequency of the sound, the mode, the brightness, and the 'on' status. The output is shown as a continuous stream of these JSON objects, separated by new lines.

```
skuska1.ino
1016  > digitalWrite(LED_BUILTIN, HIGH);
1017  FastLED.show();
1018  }
1019
1020  else if (cmd.startsWith("BRIGHTNESS:")) {
1021    int value = cmd.substring(11).toInt();
1022    value = constrain(value, 0, 255);
1023
1024    BRIGHTNESS = value;
1025    FastLED.setBrightness(BRIGHTNESS);
1026  }
1027
1028  else if (cmd.startsWith("MODE:")) {
1029    String modeValue = cmd.substring(5);
1030
1031    if (modeValue == "Sound monitor") {
1032      this_mode = 0;
1033    }
1034    else if (modeValue == "LED visualizer") {
1035      this_mode = 2;
1036    }
1037    else if (modeValue == "calibration") {
1038      fullLowPass();
1039    }
1040    else {
1041      // Default mode
1042    }
1043  }
1044
1045  // Send data back to the serial port
1046  String json = "{\"sound\":\"1023\",\"freq\":\"910\",\"mode\":\"4\",\"brightness\":\"200\",\"on\":\"1\"}";
1047  Serial.println(json);
1048  delay(200);
1049  }
```

Output Serial Monitor x

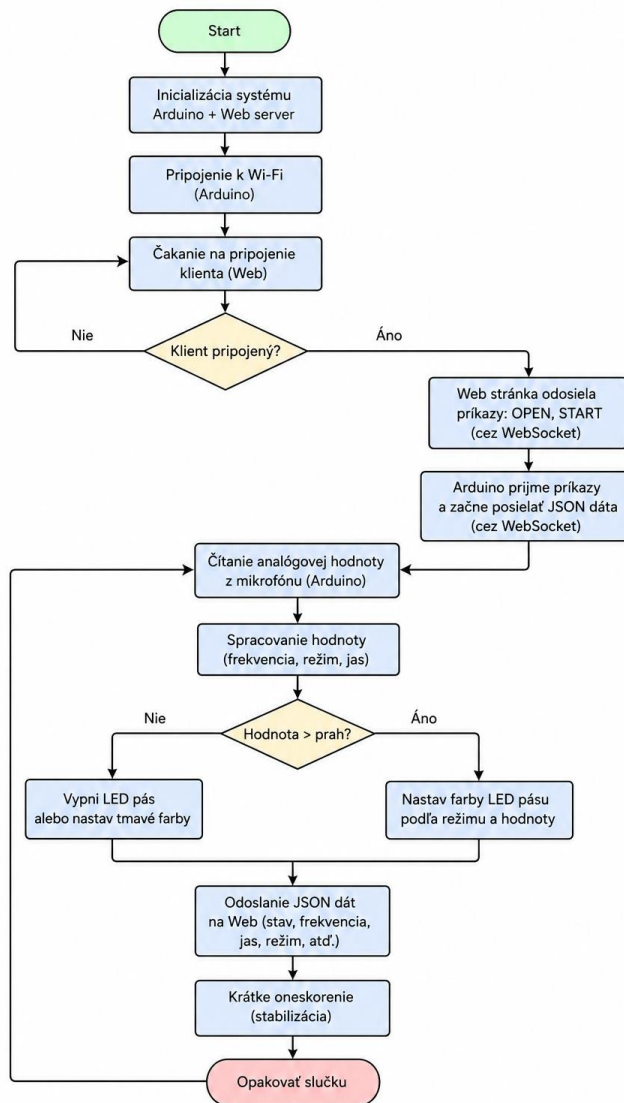
Message (Enter to send message to 'Arduino Nano' on 'COM9')

New Line 9600 baud

```
{\"sound\":\"1023\",\"freq\":\"910\",\"mode\":\"4\",\"brightness\":\"200\",\"on\":\"1\"}
{\"sound\":\"1023\",\"freq\":\"921\",\"mode\":\"4\",\"brightness\":\"200\",\"on\":\"1\"}
{\"sound\":\"1013\",\"freq\":\"906\",\"mode\":\"4\",\"brightness\":\"200\",\"on\":\"1\"}
{\"sound\":\"1008\",\"freq\":\"892\",\"mode\":\"4\",\"brightness\":\"200\",\"on\":\"1\"}
{\"sound\":\"1011\",\"freq\":\"893\",\"mode\":\"4\",\"brightness\":\"200\",\"on\":\"1\"}
{\"sound\":\"1023\",\"freq\":\"914\",\"mode\":\"4\",\"brightness\":\"200\",\"on\":\"1\"}
{\"sound\":\"1023\",\"freq\":\"927\",\"mode\":\"4\",\"brightness\":\"200\",\"on\":\"1\"}
{\"sound\":\"1023\",\"freq\":\"935\",\"mode\":\"4\",\"brightness\":\"200\",\"on\":\"1\"}
{\"sound\":\"1023\",\"freq\":\"924\",\"mode\":\"4\",\"brightness\":\"200\",\"on\":\"1\"}
{\"sound\":\"1023\",\"freq\":\"915\",\"mode\":\"4\",\"brightness\":\"200\",\"on\":\"1\"}
{\"sound\":\"1023\",\"freq\":\"947\",\"mode\":\"4\",\"brightness\":\"200\",\"on\":\"1\"}
{\"sound\":\"1016\",\"freq\":\"899\",\"mode\":\"4\",\"brightness\":\"200\",\"on\":\"1\"}
```

Serial Monitor s JSON údajmi

8. Vývojový diagram a UML



Vývojový diagram aplikácie

9. Používateľská príručka

Postup spustenia systému:

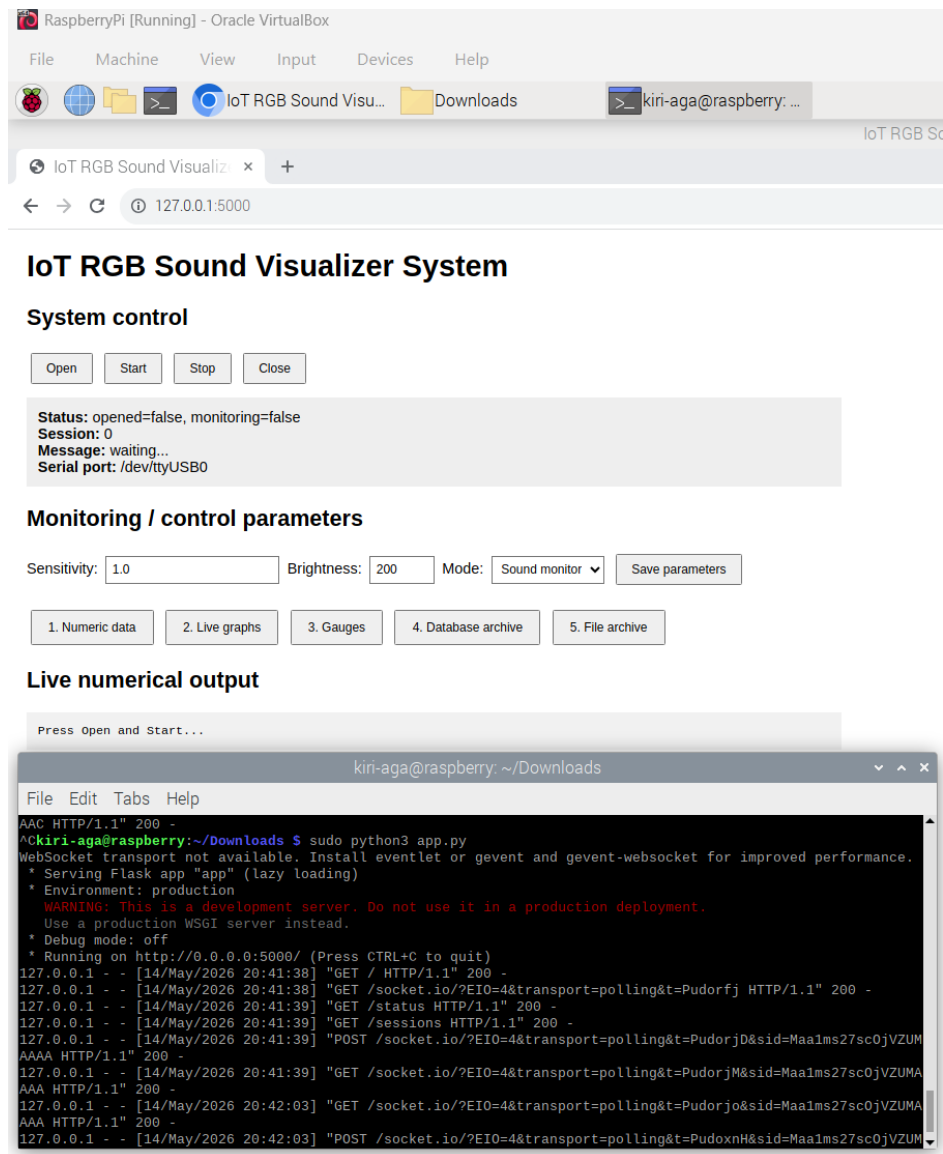
1. Spustiť Raspberry Pi OS vo VirtualBoxe.
2. Pripojiť Arduino Nano cez USB.
3. Spustiť Flask server:
`python3 app.py`

4. Otvoriť webový prehliadač:
<http://127.0.0.1:5000>

5. Kliknúť na tlačidlo Open.
6. Kliknúť na tlačidlo Start.
7. Pustiť hudbu.
8. Sledovať LED pás a grafy.
9. Po skončení kliknúť na Stop.
10. Kliknúť na Close.

Používateľ môže meniť:

- jas LED,
- režim systému,
- monitorovanie údajov.



Raspberry Pi OS vo VirtualBoxe s otvorenou aplikáciou

10. GitHub a verzionovanie

Pri vývoji projektu bol používaný verzionovací systém GitHub.

Zdrojové kódy projektu sú dostupné na GitHub:

<https://github.com/kiriagap/IoT-LED-Visualizer>

GitHub repozitár obsahuje:

- Arduino zdrojové kódy,
- Flask server,

- webové rozhranie,
- databázové súbory,
- technickú dokumentáciu.

Výhody použitia GitHub:

- verzionovanie projektu,
- archivácia zmien,
- jednoduché zdieľanie projektu,
- možnosť tímovej spolupráce.

11. Záver

Projekt úspešne splnil všetky požiadavky zadania.

Boli implementované:

- reálne senzory,
- webová IoT komunikácia,
- vizualizácia údajov,
- grafy,
- ciferníky,
- archivácia údajov,
- databáza,
- CSV export,
- serverová časť,
- klientská časť.

Projekt využíva reálny hardvér Arduino a zároveň moderné IoT princípy.

Ako základ Arduino kódu bol použitý projekt od blogera AlexGyver, ktorý bol následne upravený a rozšírený podľa požiadaviek projektu (webové rozhranie, IoT komunikácia a ďalšie funkcionality).

Možné budúce rozšírenia:

- MQTT komunikácia,
- cloudové úložisko,
- mobilná aplikácia,
- hlasové ovládanie,
- smart home integrácia.

12. Rozšírenie IoT systému pomocou služby ngrok

Po dokončení základnej verzie projektu bol systém rozšírený o možnosť vzdialeného prístupu cez internet pomocou služby ngrok. Pôvodná verzia webovej aplikácie bola dostupná iba lokálne na adrese `http://127.0.0.1:5000` v prostredí Raspberry Pi OS vo VirtualBoxe. Takéto riešenie síce umožňovalo webové ovládanie systému, ale prístup bol obmedzený iba na lokálny počítač.

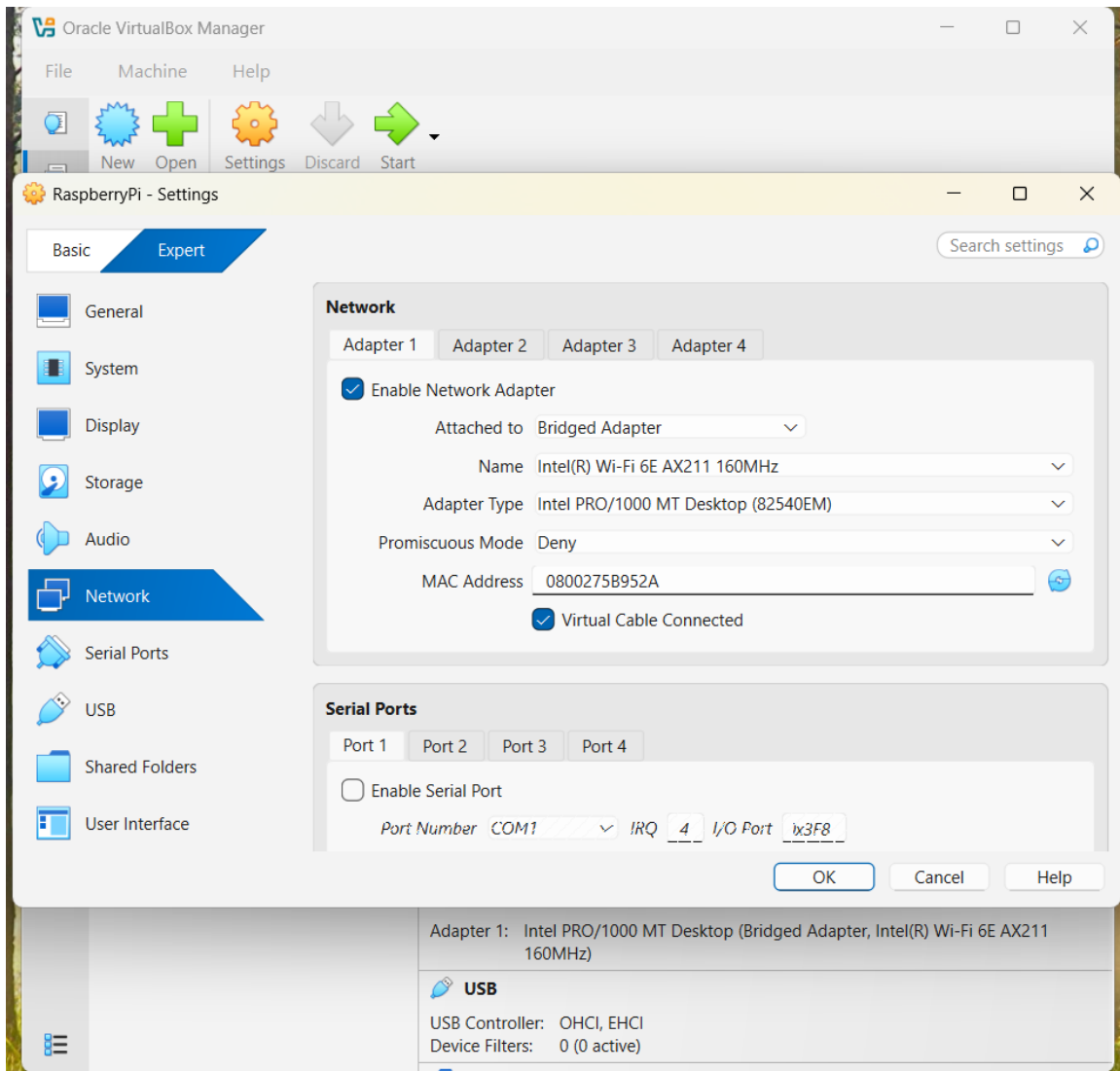
Implementáciou služby ngrok bol Flask server sprístupnený cez verejnú HTTPS adresu. To znamená, že webové rozhranie projektu je možné otvoriť aj z iného zariadenia pripojeného na internet. Projekt tak získal plnohodnotnú IoT funkcionality.

12.1 Konfigurácia VirtualBox siete

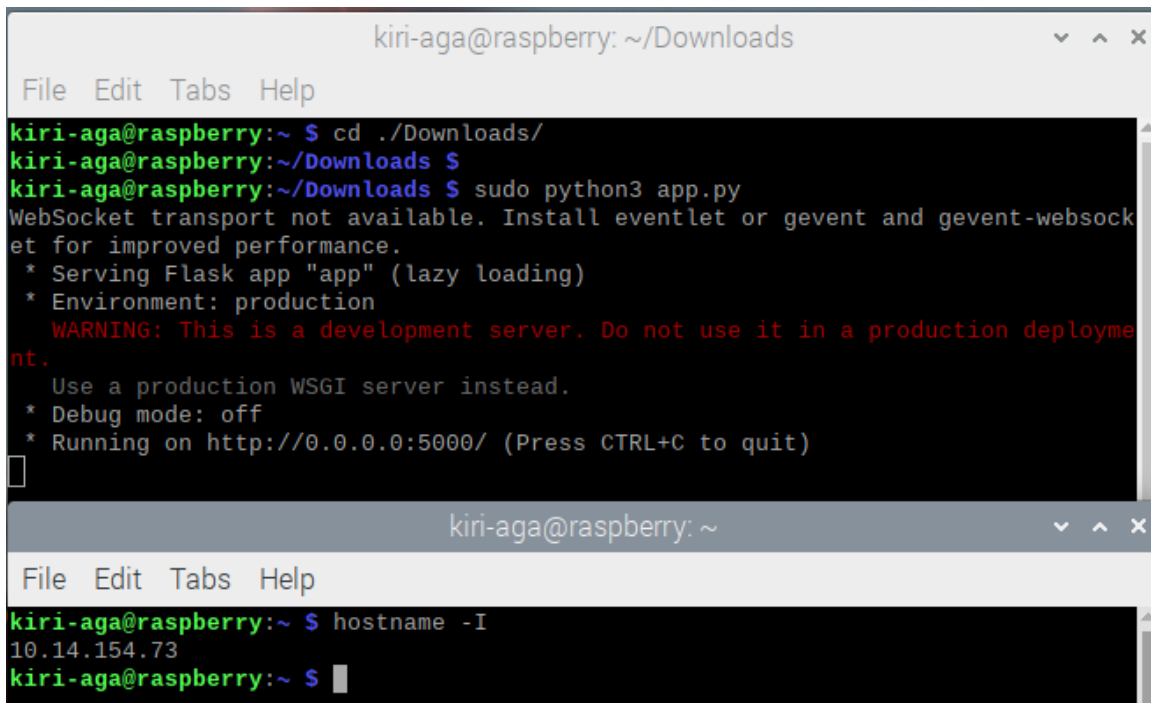
Pre správnu komunikáciu medzi Windows hostiteľom, Raspberry Pi OS a službou ngrok bolo potrebné zmeniť nastavenie siete vo VirtualBoxe. Predvolený NAT režim bol nahradený režimom Bridged Adapter.

Po zmene konfigurácie získal Raspberry Pi OS vlastnú IP adresu v lokálnej sieti, napríklad 10.14.154.73.

- Otvorenie nastavení VirtualBox.
- Výber Network → Adapter 1.
- Nastavenie režimu Bridged Adapter.
- Reštartovanie Raspberry Pi OS virtuálneho systému.
- Kontrola IP adresy pomocou príkazu `hostname -I`.



Nastavenie Bridged Adapter vo VirtualBoxe



```
kiri-aga@raspberrypi: ~/Downloads
File Edit Tabs Help
kiri-aga@raspberrypi:~ $ cd ./Downloads/
kiri-aga@raspberrypi:~/Downloads $
kiri-aga@raspberrypi:~/Downloads $ sudo python3 app.py
WebSocket transport not available. Install eventlet or gevent and gevent-websocket
for improved performance.
* Serving Flask app "app" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)

kiri-aga@raspberrypi: ~
File Edit Tabs Help
kiri-aga@raspberrypi:~ $ hostname -I
10.14.154.73
kiri-aga@raspberrypi:~ $
```

Terminál Raspberry Pi s príkazom hostname -I

12.2 Spustenie Flask servera

Webová aplikácia bola spúšťaná pomocou Flask frameworku v Raspberry Pi OS. Server bol nakonfigurovaný tak, aby počúval na všetkých sieťových rozhraniach.

Týmto spôsobom bolo možné pristupovať k webovej aplikácii aj z hostiteľského Windows systému pomocou IP adresy virtuálneho systému.

Server bol spúšťaný pomocou príkazu:

```
socketio.run(app, host="0.0.0.0", port=5000)
```

```
sudo python3 app.py
```

12.3 Sprístupnenie aplikácie cez internet pomocou ngrok

Na Windows hostiteľskom systéme bola nainštalovaná služba ngrok. Ngrok vytvoril zabezpečený HTTPS tunel medzi verejnou internetovou adresou a Flask serverom bežiacim v Raspberry Pi OS.

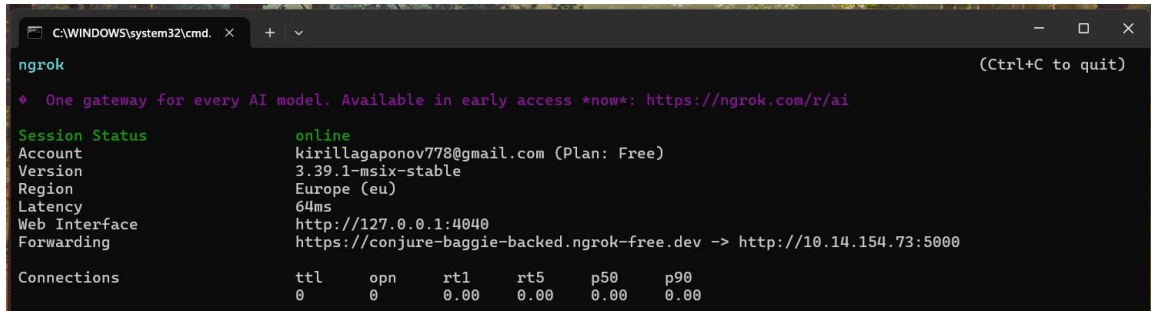
Po registrácii do služby ngrok bol nastavený autentifikačný token pomocou príkazu:

```
ngrok config add-authtoken TOKEN
```

Následne bol vytvorený HTTP tunel:

```
ngrok http 10.14.154.73:5000
```

Ngrok následne vytvoril verejnú HTTPS adresu, cez ktorú bolo možné ovládať a monitorovať systém z internetu.



```
C:\WINDOWS\system32\cmd. x + v
ngrok (Ctrl+C to quit)
One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai
Session Status online
Account kirillagaponov778@gmail.com (Plan: Free)
Version 3.39.1-msix-stable
Region Europe (eu)
Latency 64ms
Web Interface http://127.0.0.1:4040
Forwarding https://conjure-baggie-backed.ngrok-free.dev -> http://10.14.154.73:5000
Connections
tll opn rt1 rt5 p50 p90
0 0 0.00 0.00 0.00 0.00
```

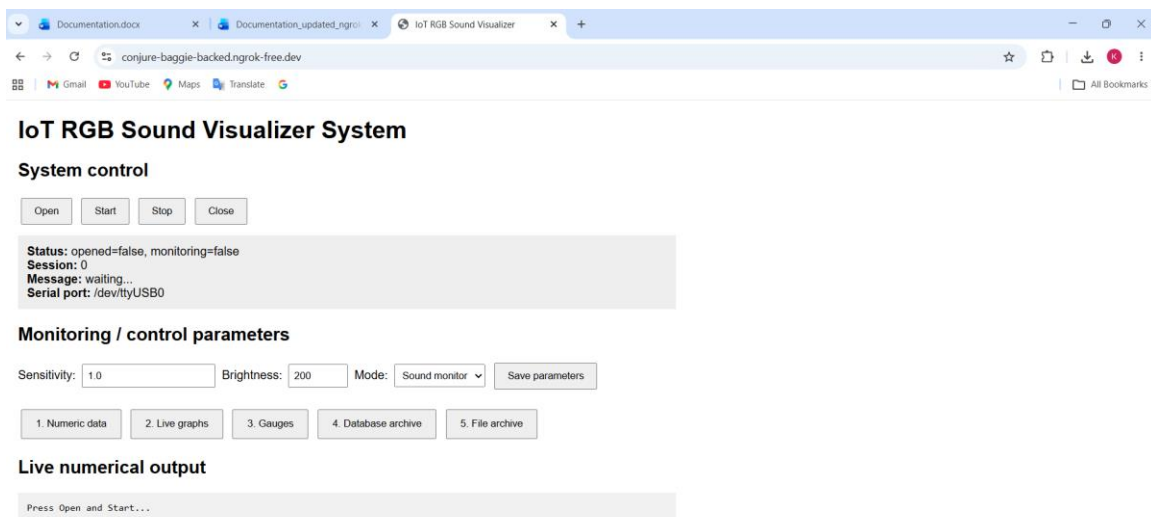
VLOŽIŤ SCREENSHOT: CMD okno s úspešne spusteným ngrok

12.4 Výsledok implementácie ngrok

Implementáciou služby ngrok sa projekt stal plnohodnotným IoT systémom s možnosťou vzdialeného monitorovania a ovládania cez internet. Používateľ môže pristupovať k webovej aplikácii z iného počítača alebo mobilného zariadenia.

- vzdialené monitorovanie systému
- vzdialené ovládanie LED systému
- prístup cez HTTPS protokol
- jednoduchá prezentácia projektu
- rozšírenie IoT funkcionality projektu
- možnosť ďalšieho cloudového rozšírenia

Táto funkcionality významne rozšírila možnosti projektu a zároveň preukázala praktické využitie IoT technológií v kombinácii s reálnym hardvérom, webovou aplikáciou a vzdialenou komunikáciou.



Otvorená IoT webová aplikácia cez ngrok URL